Routers, Forwarding, and Packet Scheduling

Lecture 20

Computer Networks and Distributed Programming

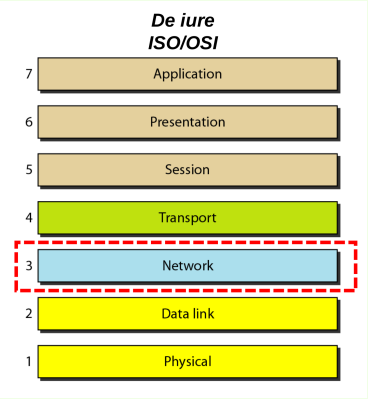
Where we are: the box where packets are lost

For three lectures the senders have been reacting to something they could not see.

- A timeout, three duplicate acknowledgements, a window that closes: all of it is a reaction to a loss.
- And the loss itself was a curve — queueing delay rising as the traffic intensity approaches one, and then packets dropped.
- Nobody ever opened the node where that happens. It is a machine with links coming in and links going out.
- Below the transport layer there are no processes any more: this layer moves datagrams from host to host, and a piece of it runs in every router on the path.
- Two questions: how does a datagram arriving on one link leave on the right one, and who decides which packet goes first.

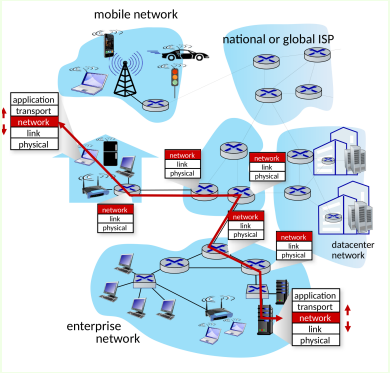
From segments to datagrams

- At the sending host: take a segment from the transport layer, encapsulate it into a datagram, send it into the network.
- At the receiving host: take the datagrams that arrive, extract the segments, deliver them up.
- Between the two, nothing of what we have studied so far applies: no connection, no acknowledgement, no window.
- Some of the protocols this course still owes you live here: IP first of all, and then DHCP, NAT, and the rest.
- And here live the devices that make the network a network: routers.



A truncated stack at every hop

- Inside the network there are nodes that forward datagrams to adjacent nodes, one step at a time, until the destination.
- Each such redirection is called a hop.
- The routers are drawn with a truncated stack: they have no application layer and no transport layer, and they run neither.
- Transport software runs only at the edges. This layer runs everywhere — in both hosts and in every router between them.
- That is why the network layer is the one place where the whole Internet has to agree.



Services the network layer could offer

A layer that carries packets from host to host could promise rather more than that.

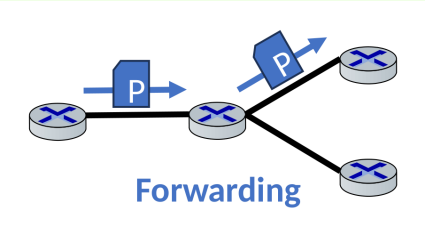
- Guaranteed delivery: a packet sent by the source will eventually arrive at the destination.
- Bounded delay: not only delivered, but delivered within a stated bound — within 100 ms, say.
- In-order delivery: packets arrive in the order in which they were sent.
- Minimal bandwidth: the sender names a rate, say 1 Mbps, and as long as it stays below it nothing is dropped.
- Security: datagrams encrypted at the source and decrypted at the destination, for every application at once.

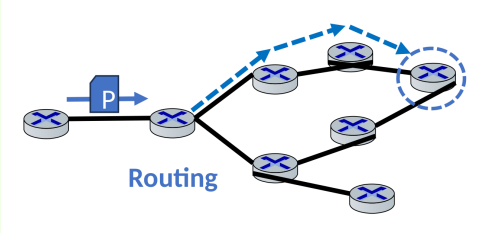
Best-effort is the only one on offer

- The Internet's network layer provides one service: it tries its best to move a packet from source to destination.
- No order, no delivery, no delay bound, no bandwidth. Every one of the promises on the previous slide is absent.
- It reads like a euphemism for no service at all: a network that delivered nothing would satisfy the definition.
- Other architectures did more — ATM guaranteed in-order delivery, bounded delay and a minimum rate — and the Internet did not follow.
- With enough bandwidth and applications that adapt, it has proven good enough for streaming video, voice and video over IP, and conferencing.
- Everything reliable you have written this term is built on top of this, by the two ends, out of nothing.

Forwarding and routing

- Forwarding: a packet arrives on an input link and the router moves it to the appropriate output link. One node, one packet, one decision.
- Routing: deciding the path that packets take from sender to receiver. The procedures that compute it are the routing algorithms.
- A router can also do more than move a packet: block it, drop it if it has expired, duplicate it on several links, or rewrite part of it.
- Driving to another city: forwarding is getting through one interchange, routing is having chosen the route before leaving.



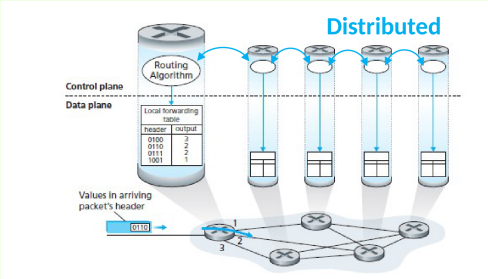


Nanoseconds and seconds: two planes

- Forwarding is router-local: input interface to output interface, inside one box. It takes a few nanoseconds, so it is done in hardware.
- Routing is network-wide: it settles the end-to-end paths. It takes seconds, so it is done in software.
- Two functions on timescales a billion apart, which is why they are named separately: data plane: forwarding, control plane: routing.
- Between them sits one data structure, the forwarding table: the control plane writes it, the data plane reads it once per packet.
- Consequence: a router forwards without knowing the path. It knows only which of its own links is next.

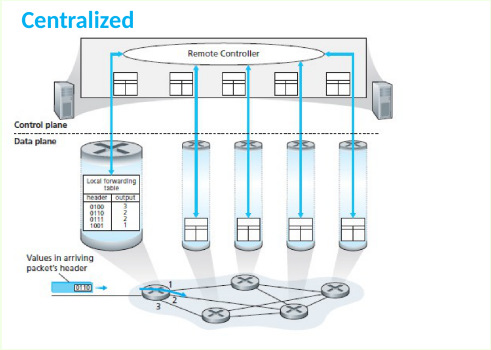
Where the table comes from

- A router forwards by examining one or more header fields and using those bits to index into its table.
- The entry it lands on names the outgoing interface, and that is the whole of the decision.
- Traditionally the table is computed inside the router, by a routing algorithm talking to those in the other routers.
- They exchange routing messages; each router ends up with its own local table, and nobody holds the whole picture.
- How they agree is a subject of its own, and we will return to it.



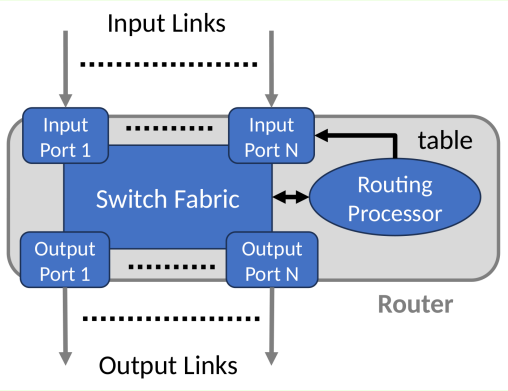
One controller for all the tables

- The other answer: a physically separate controller computes the tables and hands them out, and the routers only forward.
- It may sit in a remote data centre, built for redundancy, run by the ISP or by a third party.
- This is software-defined networking: the thing computing the tables is a program, and several are open.
- Note what does *not* change: the data plane of the two pictures is identical. Only the author of the table differs.



Four components

- Input ports: where a link ends, and where the table is consulted.
- Switching fabric: what connects the input ports to the output ports: a network inside the router.
- Output ports: they hold what comes out of the fabric and transmit it on the outgoing link.
- Routing processor: it runs the routing protocols, keeps the state of the links, and computes the table.
- The first three are hardware; the fourth is an ordinary CPU. The arrow between them carries the table.



What an input port does

- It terminates the physical link and does the link-layer work needed to talk to the other end of that link.
- It performs the lookup: it consults the forwarding table and prepares the fabric for the output port it has chosen.
- It sends control packets — those carrying routing information — up to the routing processor instead of across.
- It also checks the version, the checksum and the time-to-live, rewrites the last two, and updates the management counters.
- Each port keeps its own copy of the table, received over a dedicated bus, so no packet has to wait for a central processor.
- Careful: these ports are physical interfaces. They have nothing to do with the sixteen-bit numbers of the transport layer.

Why lookup cannot be software

- Take a 100 Gbps input link and a 64-byte datagram. The port has 5.12 ns before the next one may arrive.
- Line cards carry several ports at once, and then the whole pipeline has to run N times faster than that.
- No software gets near those figures, which is why input ports, output ports and fabric are all hardware — the vendor's own design or bought-in silicon.
- The routing processor lives at the other end of the scale, on milliseconds and seconds, and can be an ordinary CPU running ordinary code.
- The shape of the layer: the fast part is dumb and repetitive, the clever part is slow and rare.

Edge routers and core routers

- Home and small business: a handful of ports, one link towards the ISP, and the whole router in a plastic box.
- Edge routers: they join one or more networks to another, typically a network to its provider. This is the common case.
- Core routers: they move packets *within* one network, and they are what the backbone of the Internet is made of.
- Scale is the difference, not function: the Juniper MX2020 supports up to 800 ports of 100 Gbps, for 800 Tbps in one chassis.


Tp-link AX6600
home router

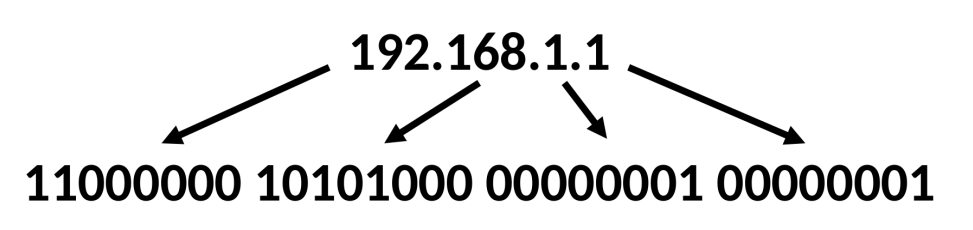

Cisco RV016-G5
business router


Juniper MX2020,
edge router


Cisco CRS-1
backbone core router

The table maps addresses to ports

- The job of the forwarding table is to associate addresses with output ports, so that the packet leaves on the link that leads to the next node.
- An address is four bytes, 32 bits. We write it in decimal for our own convenience; the router never sees that form.
- So the lookup is a comparison of bits, and everything that follows is a question of which bits are compared.



One row per address is out of the question

Address	Output port
128.37.42.211	1
137.201.47.103	2
⋮	⋮



What is wrong with simply listing the addresses?

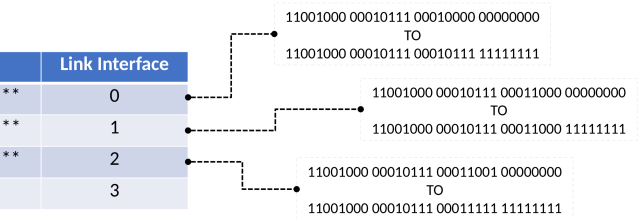
One row for every possible destination is 2^{32} rows — over four billion — in *every* router, to be searched in nanoseconds. And it would have to be rewritten whenever a host anywhere appeared or vanished. So a row does not name an address: it names a group of addresses that share their first bits, a subnet.

A row is a prefix, and a prefix is a range

- Each row fixes the first bits and leaves the rest free. The fixed part names the group; the free part names a host inside it.
- The first row fixes 21 bits. The 11 free ones give $2^{11} = 2048$ addresses, from all-zeros to all-ones — the two ends of the range on the right.
- So a row is an interval, and four rows replace four billion.
- The last row, `otherwise`, is the default: whatever matched nothing still has to leave somewhere.

Example of a 4-ports table

Prefix of IP addresses	Link Interface
11001000 00010111 00010*** ********	0
11001000 00010111 00011000 ********	1
11001000 00010111 00011*** ********	2
otherwise	3

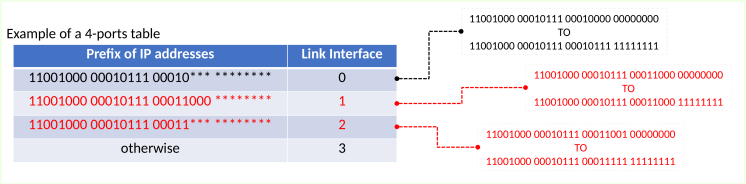


Only the prefix bits are compared

Table entry	11001000 00010111 00011000 ********
Destination	11001000 00010111 00011000 11001001

- The comparison runs over the prefix only and stops where the prefix stops. Here that is 24 bits, and they agree: a match.
- The eight bits left over say *which* host inside the group, and this router has no use for that.
- Which is the point of the whole arrangement: a router in another continent needs to know the group, never the machine.

Two rows can both match

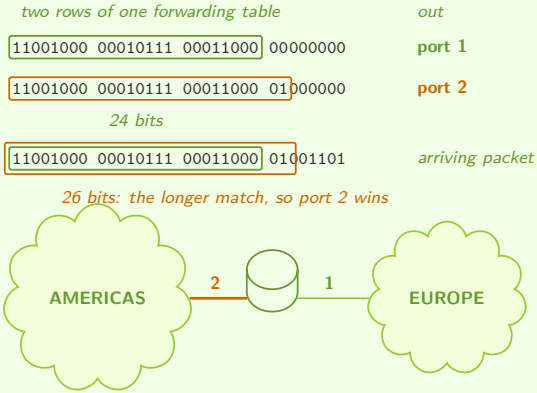


Longest prefix matching

When an address matches more than one row, the packet is forwarded to the interface of the row with the longest prefix.

The ranges on the right are the ones *after* the rule has been applied: that is why the third begins at ...00011001.

Carving a block out of a block



Both rows describe the same 24-bit block; the second takes a quarter of it — two bits more — and sends that quarter elsewhere. Without the rule the two would contradict each other; with it, they say this whole block that way, except this piece. It is how one customer is moved without renumbering anybody.

Lookup in nanoseconds

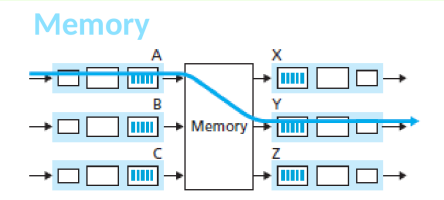
- Stated as a problem it is simple: find the longest matching entry. At gigabit rates it has to be answered in nanoseconds.
- So not only in hardware, but with more than a linear scan: the search through a large table is an engineering subject of its own.
- Memory access time is the constraint that shapes the design — DRAM embedded on the chip, faster SRAM used as a cache in front.
- TCAM: present the 32 bits to the memory and it returns the entry in essentially constant time; large devices hold on the order of a million entries.
- Once the output port is known the packet goes into the fabric — or waits at the input, if other ports are using it.

Match plus action

- Looking up a destination (match) and then sending the packet to the chosen port (action) is a pattern, not a peculiarity of routers.
- Link-layer switches: the same two steps, on a different address in a different header.
- Firewalls: the match is a combination of addresses and transport port numbers; the action is to drop.
- Network address translators: the match is a port number; the action is to rewrite it before forwarding.
- Three devices this course meets separately, doing one thing in three costumes. It is worth recognising when they turn up.

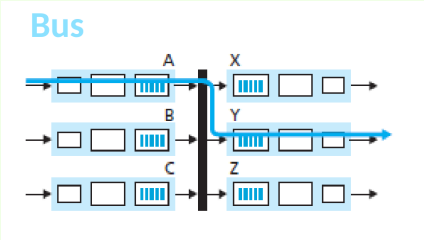
Switching via memory

- The earliest routers were ordinary computers, and the ports were ordinary I/O devices.
- An arriving packet interrupted the processor, was copied into memory, looked up, and copied out to the output port's buffer.
- The packet crosses memory twice. If memory can take B packets per second, the router forwards fewer than $B/2$.
- And two packets cannot be forwarded at once even to different ports: one read or write at a time on the shared bus.
- Modern shared-memory routers exist, but the lookup and the write happen on the line card, not in one central CPU.



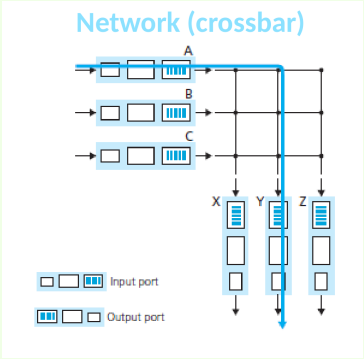
Switching via a bus

- The input port hands the packet straight to the output port over a shared bus, with no routing processor in between.
- It prepends an internal label naming the output port. All ports see the packet; only the one named keeps it, and it strips the label.
- One packet at a time on the bus: the switching speed of the router is the speed of the bus.
- Which is often enough. A well-known enterprise router of this kind switches over a 32 Gbps backplane.
- Simple, cheap, and a hard ceiling.



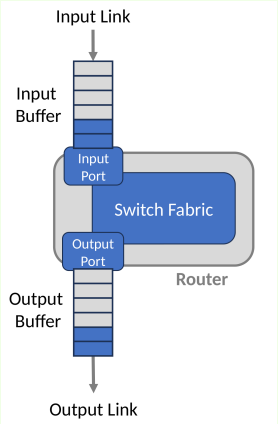
Switching via an interconnection network

- A crossbar: $2N$ buses joining N inputs to N outputs, with a crosspoint at each intersection that opens and closes.
- To move a packet from A to Y the controller closes the crosspoint AY, and only bus Y picks it up.
- So $A \to Y$ and $B \to X$ happen at the same time: packets in parallel, which the other two cannot do.
- Non-blocking: a packet is never stopped on its way to an output port that nobody else is using.
- But two packets for the same port still contend, and one waits at the input. Remember that.



Queues at both ends of the fabric

- Switching takes time, so both input and output ports hold packets in buffers while they wait — cars at a traffic light.
- How much queueing there is is not fixed: it depends on the traffic load, on the speed of the fabric, and on the line speed.
- Packets can arrive faster than the fabric moves them, and leave the fabric faster than the link carries them away. Either way the buffer grows, and it can overflow.
- So: this is the queue behind the queueing delay we drew as a curve, and this buffer is the loss that a sender can only infer.

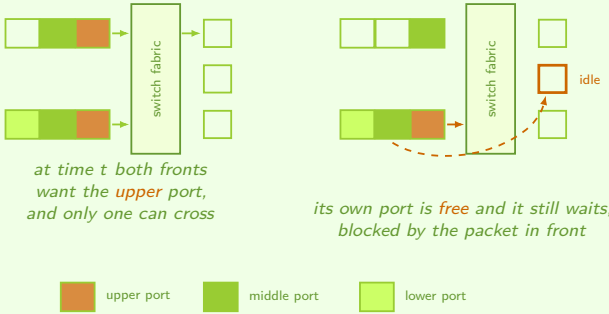


How fast must the fabric be

N input ports, N output ports, every line at R_{line}, and the worst case: everything arriving is destined for the same output port.

- $R_{switch} \approx N\,R_{line}$: input queueing is negligible — a batch of N clears before the next arrives — but the output port receives N times what it can send.
- $R_{line} < R_{switch} < N\,R_{line}$: queues at both ends: the input waits for the fabric, the output waits for the link.
- $R_{switch} \approx R_{line}$: nothing much queues at the output, because little gets through; the queues are at the input.
- Read the three together: no choice of fabric removes the queueing. It only decides where it happens.

The packet that waits for a free port



The fabric is non-blocking. So why is a packet stopped?

Because an input queue is served in order: the packet at the head is waiting for a busy output port, and everything behind it waits too — including a packet whose own port is idle. This alone makes the input queue grow without bound once the arrival rate reaches about 58% of the line capacity.

When there is no room left

- Even with a fabric N times faster than the lines, N packets can reach one output port in the time it takes to send one. Output queues form anyway.
- When the memory is gone there are two choices: drop the arriving packet — drop-tail — or make room by removing packets already queued.
- A packet leaves the queue only once it has been completely transmitted. Until then it occupies its buffer.
- It can also pay to drop, or to mark, *before* the buffer is full, as a warning to the sender: these are the active queue management policies, RED the best known of them.
- And marking rather than dropping is exactly what those two bits in the header were for — this is the machine that sets them.

How much buffering is enough

- The rule of thumb for years: buffer = average round-trip time × link capacity. A 10 Gbps link at 250 ms wants 2.5 Gbit of buffer.
- That figure came from analysing a small number of TCP flows. With N large and independent, the requirement falls to RTT $\times\ C/\sqrt{N}$.
- In the core, where thousands of flows share a link, that is a very large reduction.
- And more is not better: a bigger buffer absorbs larger bursts *and* lengthens the queueing delay of everything inside it.
- Ten times the buffer to avoid loss can mean ten times the end-to-end delay — and a longer round trip makes every sender on that path slower to react.

The queue that never empties

One flow, no other traffic, and a delay that never falls

A home router takes 20 ms to send a packet, the round trip is 200 ms, and at $t = 0$ a burst of 25 packets arrives at the queue.

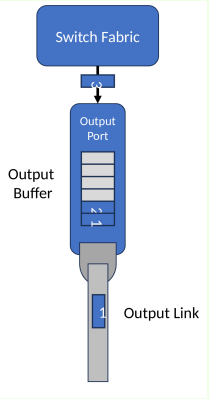
- At $t = 200$ ms the first acknowledgement arrives, just as the 21st packet goes out — and it releases one new packet into the queue.
- At $t = 220$ the next one arrives, and another packet is queued. One in for every one out, for as long as the transfer lasts.
- The queue settles at five packets and stays there. The pipe is full, the throughput is exactly right, and the delay is constant, large, and caused by nothing but the buffer.
- Bufferbloat: nobody is misbehaving. The sender is doing what it was designed to do, and the buffer is what is wrong.

Choosing which packet goes next

- Several packets, possibly from several input ports, are waiting for one outgoing link. Something has to choose the order.
- First-come-first-served: in order of arrival, and nothing else. Also called FIFO.
- Priority queuing: the packets are sorted into classes, and the more important class goes first.
- Round robin: classes again, but served in turn instead of by rank.
- Whichever it is, a packet is removed only when it has been fully transmitted: the choice is made at the end of each transmission, not continuously.

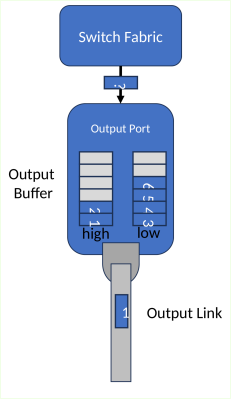
FIFO

- One queue. Packets are selected for transmission in the same order in which they arrived at the output port.
- If the link is busy, an arriving packet is buffered; if there is no room for it, the discarding policy decides its fate.
- No packet can be favoured, and none can be delayed for another's benefit. Everything on the link shares one fate.
- Which is the reason it is the default: there is nothing to configure and nothing to get wrong.



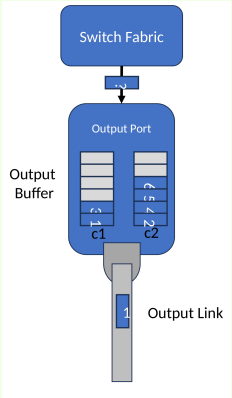
Priority queuing

- Arriving packets are classified into priority classes, typically by the transport port numbers they carry.
- One queue per class. The scheduler transmits from the highest-priority non-empty queue, and inside a class it is FIFO.
- An operator may put network management traffic, or real-time voice, ahead of mail.
- Non-preemptive: a transmission already begun is never interrupted. A high-priority packet arriving mid-transmission still waits for that packet to finish.

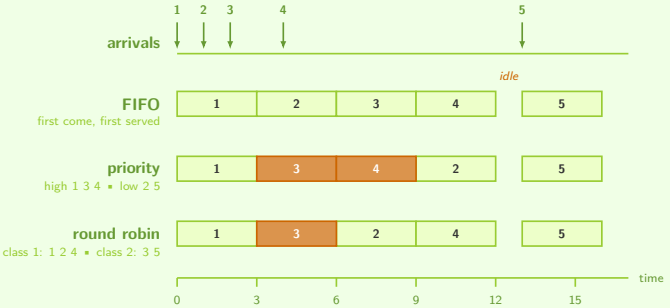


Round robin and weighted fair queuing

- The same classification, without the rank: the scheduler alternates among the classes, one packet each in turn.
- Work-conserving: it never leaves the link idle. A class with an empty queue is skipped at once.
- WFQ gives each class i a weight w_i, and a class with packets to send receives a share $w_i / \sum_j w_j$ of the service.
- So on a link of rate R a class is guaranteed at least $R \cdot w_i / \sum_j w_j$, whatever the others do — and strict priority guarantees the low class nothing at all.



Same arrivals, three departure orders



- Priority: 3 and 4 are high, so both overtake 2 — which arrived before either and leaves last of the four.
- Round robin: 3 goes early because its class was owed a turn; 4 does not overtake 2, being of the same class.
- The link is idle before 5 in all three: a scheduler decides an order, it never creates capacity.

Who decides what a class is

- The mechanism is neutral. What counts as a class is decided by whoever runs the network, on any field of the header.
- It can be a well-known port — management ahead of mail — or a source address: the traffic of a company that has paid, ahead of one that has not.
- The same match-plus-action can also block traffic from a given company, or from a given country.
- So the limits here are legal rather than technical, and they differ by country. The American rules of 2015 forbade blocking, throttling and paid prioritisation; they were replaced in 2017 by transparency requirements.
- The question returns from the money's side when we ask why two networks agree to carry each other's traffic at all.

Summary

- The network layer offers one service, best-effort. Order, delivery and delay guarantees are absent, and the ends built what they needed on top.
- Forwarding is local and takes nanoseconds; routing is network-wide and takes seconds; the forwarding table is where the two meet.
- A row is a prefix, that is a group of addresses sharing their first bits — and when several rows match, the longest one wins.
- Three fabrics — memory, bus, crossbar. The last forwards in parallel and still cannot help two packets that want the same output port.
- Queues form wherever a rate meets a slower one, at the input, at the output, or both; that is where the loss your senders infer is actually made.
- Scheduling settles the order. What is still missing is where the addresses inside those prefixes come from, and that is next.